Deloitte.

CBO Academy

Clean Code Guidelines

(Code Health)

Document Control Information

Document Information

Document Identification	Clean Code Guidelines
Document Name	Clean Code Guidelines
Project Name	CBO Academy
Document Author	Pooja Modi
Document Version	V1
Document Status	Draft
Date Released	05-JUL-2023

Document Edit History

Version	Date	Additions/Modifications	Prepared/Revised by
V2	18-AUG-2025	Update Exception handling guidelines	Reshmi Sasidharan

Document Review/Approval History

Date	Name	Organization/Title	Comments
< dd-mmm-yyyy >	<Name>	<Organization/Title>	<Comments>

Distribution of Final Document

The following people are designated recipients of the final version of this document:

Name	Organization/Title
<Name>	<Organization/Title>

Contents

1 Introduction

1.1 Purpose

The purpose of this document is to provide project team a set of standard frameworks to be considered during development and Java coding standards to follow to produce clean and consistent code while developing the project CBO Academy.

1.2 Scope

The scope of this document provides the frameworks, conventions, guidelines, and best practices of application software code development which promotes the consistency and quality of the software code.

1.3 Java Version

We recommend using Java version 11 and Spring Boot version: 2.7.13 for CBO Academy.

2 Java 11

2.1 Project Structure

Maven and Gradle are both build automation tools that help you manage your project structure. They allow you to define the dependencies of your project, as well as the tasks needed to compile, test, package, and deploy it.

Using Maven or Gradle helps with the code organization. By defining the directory structure for the project, we can ensure that related files are easily accessible and kept together.

We will be using Gradle for CBO Academy.

Below is an example for sample folder structure for a project :

- *src/main/java*: For source files
- *src/main/resources*: For resource files, like properties
- *src/test/java*: For test source files
- *src/test/resources*: For test resource files, like properties

2.1.1 Using the "default" Package

When a class does not include a package declaration, it is in the "default package". The use of the "default package" is generally discouraged and should be avoided.

2.1.2 Use Configuration Files

Instead of hardcoding the username and password, consider putting them in a configuration file that the application reads at runtime. Java provides multiple libraries to load and manage configurations, such as properties files, yaml files, or JSON files.

Configuration files can be stored outside the project folder, making maintenance more accessible and safer.

2.1.3 Use Environment Variables

Another best option to store confidential information is through environment variables. This option is provided by most operating systems, allowing you to set variables that can be accessed by any application running on the system. For example, storing your user parameters in environment variables is a much safer and more efficient way of managing user parameters.

2.1.4 Domain

We recommend that you follow Java's recommended package naming conventions and use a reversed domain name (for example, com.example.project).

2.1.5 Models and DTOs

The various models of the application are organized under the *model* package, their DTOs(data transfer objects) are present under the *dto* package.

2.1.6 Avoid Hardcoding

Hardcoding is the practice of adding the raw data directly into the source code of a program wherever needed instead of using a variable to store the raw data and using that variable. It can cause serious maintainability problems in your project.

Example:

- Let's say you have decided to use 404 as the error code in your project. Whenever, an error occurs in a method 404 is returned. Using this error code 404, the control flow is changed as numerous places.
- Now, what happens when you need to use a new error code say -1. You have replaced the raw value 404 with -1 at all the places where it is used.
- This is very difficult and irritating when it comes to large projects. Instead, we can simply use a constant ERROR_CODE and assign it the value 404 or -1. Now, this constant identifier can be used anywhere in the code.
- If, in future, any change is required, we must only change the ERROR_CODE constant identifier and rest everything will work smoothly.

2.1.7 Comments

- Java programs can have two kinds of comments: documentation comments and implementation comments.
- Documentation comments (known as "Javadoc comments") are Java-only, and are delimited by /**...*/. Javadoc comments are meant to describe the specification of the code, from an implementation-free perspective to be read by developers who might not necessarily have the source code at hand.
- Implementation comments are those found within the code, which are delimited by /*...*/, and //. Implementation comments are comments about the particular implementation or are meant for commenting out code.
- When more than two lines are being commented, /* .. */ is preferred. When two or fewer lines are being commented, // is preferred.
- Make sure code comments are clear and concise.
- Make sure the comment is necessary or helpful.
- Java code, for the most part, should be self- documenting so the use of comments can be kept to a minimum.
- Implementation comments can be **block comment, single-line comment, trailing comment** or **comment to take out code**. When you feel compelled to add a comment, consider rewriting the code to make it clearer. Comments should **not** be enclosed in large boxes drawn with asterisks or other characters. Comments should **never** include special characters such as form-feed and backspace.

2.1.8 Don't Repeat Yourself (DRY)

- Avoid code duplication by extracting common functionality into reusable methods or classes.
- Encapsulate reusable code into separate modules, libraries, or utility classes to promote code reuse and maintainability.

2.1.9 Continuous Refactoring using SonarLint

- Regularly review and refactor your code to eliminate code smells, improve code structure, and enhance maintainability with the help of SonarLint plugin
- Identify areas for improvement & simplify complex code to make the code more robust and flexible.

2.1.10 Single Responsibility Principle (SRP)

- Aim for classes and methods to have a single responsibility. This promotes modularity, improves code maintainability, and makes code easier to understand.
- Avoid writing large, monolithic classes or methods that perform multiple unrelated tasks.

2.1.11 Package Structure

The packages should be organized in a way that promotes modularity, maintainability, and separation of concerns. While there is no strict rule on how to structure packages in a Spring Boot project, a recommended used package structure is as follows:

2.1.11.1 Main Application Class Package:

The main application class, usually annotated with **@SpringBootApplication**, is usually placed in the root package of the project. This package often takes the form of the company domain name in reverse, like com.example.myapp.

```
com
└── example
    └── myapp
        └── MySpringBootApplication.java (Main Application Class)
```

2.1.11.2 Controller Package:

The controllers, responsible for handling incoming HTTP requests and returning responses, are often placed in a separate package called controller or controllers.

```
com
└── example
    └── myapp
        ├── controller
        │   └── MyController.java
        └── MySpringBootApplication.java (Main Application Class)
```

2.1.11.3 Service Package:

The service layer, where the business logic resides, is usually placed in a package called service or services.

```
com
└── example
    └── myapp
        ├── controller
        │   └── MyController.java
        ├── service
        │   └── MyService.java
        └── MySpringBootApplication.java (Main Application Class)
```

2.1.11.4 Repository Package:

The repository package contains classes responsible for data access, typically using Spring Data JPA or other data access technologies. It is often named repository or repositories.

```
com
└── example
    └── myapp
        ├── controller
        │   └── MyController.java
        ├── service
        │   └── MyService.java
        ├── repository
        │   └── MyEntityRepository.java
        └── MySpringBootApplication.java (Main Application Class)
```

2.1.11.5 Model or Entity Package:

This package contains domain model classes, entities (if using JPA), and other data-related classes.

```
com
└── example
    └── myapp
        ├── controller
        │   └── MyController.java
        ├── service
```

```
│   └── MyService.java
├── repository
│   └── MyEntityRepository.java
├── model
│   └── MyEntity.java
└── MySpringBootApplication.java (Main Application Class)
```

2.1.11.6 Configuration Package:

Custom configuration classes, which may define beans or other configuration elements, are often placed in a package called config or configuration.

```
com
└── example
    └── myapp
        ├── controller
        │   └── MyController.java
        ├── service
        │   └── MyService.java
        ├── repository
        │   └── MyEntityRepository.java
        ├── model
        │   └── MyEntity.java
        ├── config
        │   └── MyConfig.java
        └── MySpringBootApplication.java (Main Application Class)
```

2.1.11.7 Utility Package:

Utility classes, containing helper methods or reusable code, are often placed in a package called util or utils.

```
com
└── example
    └── myapp
        ├── controller
        │   └── MyController.java
```

```
├── service
│   └── MyService.java
├── repository
│   └── MyEntityRepository.java
├── model
│   └── MyEntity.java
├── config
│   └── MyConfig.java
└── util
    └── MyUtilityClass.java
```

2.2 Naming Convention

Naming conventions make programs more understandable by making them easier to read. They can also give information about the function of the identifier (e.g., whether it is a constant, package, or class) which can clarify the code.

Identifier Type	Rules for Naming	Examples
Packages	The prefix of a unique package name is always written in all-lowercase ASCII letters and should be one of the top-level domain names, currently com, edu, gov, mil, net, org, or one of the English two-letter codes identifying countries as specified in ISO Standard 3166, 1981. Subsequent components of the package name vary according to an organization's own internal naming conventions. Such conventions might specify that certain directory name components be division, department, project, machine, or login names.	com.sun.eng com.apple.quicktime.v2
Class	Class names should be nouns, in mixed case with the first letter of each internal word capitalized. Try to keep your class names simple and descriptive. Use whole words-avoid acronyms and abbreviations (unless the abbreviation is much more widely used than the long form, such as URL or HTML).	class Customer class CaseInitiation
Interfaces	Interface names should be capitalized like class names.	interface RasterDelegate interface Storing
Methods	Methods should be verbs, in mixed case with the first letter lowercase, with the first letter of each internal word capitalized.	run() runFast() getBackground()
Variables	Variables names should be mixed or proper case starting with a lower case first letter. They should not start with underscore (_) or dollar sign ($) characters, even though they are both allowed. They should be short yet meaningful. Variable names should be short yet meaningful. The choice of a variable name should be mnemonic- that is,	int count; char grade; float myWidth;

Identifier Type	Rules for Naming	Examples
	designed to indicate to the casual observer the intent of its use. One-character variable names should be avoided except for temporary "throwaway" variables (e.g. variables used within for loops). Common names for temporary variables are i, j, k, m, and n for integers; c, d, and e for characters.	
Constants	The names of variables declared class constants and of ANSI constants should be all uppercase with words separated by underscores ("_"). (ANSI constants should be avoided, for ease of debugging.) Interface constants must always be public, static, and final.	static final int MIN_WIDTH = 4 static final int MAX_WIDTH = 999 static final int GET_THE_CPU = 1

Table 3: Naming Conventions

2.3 EditorConfig

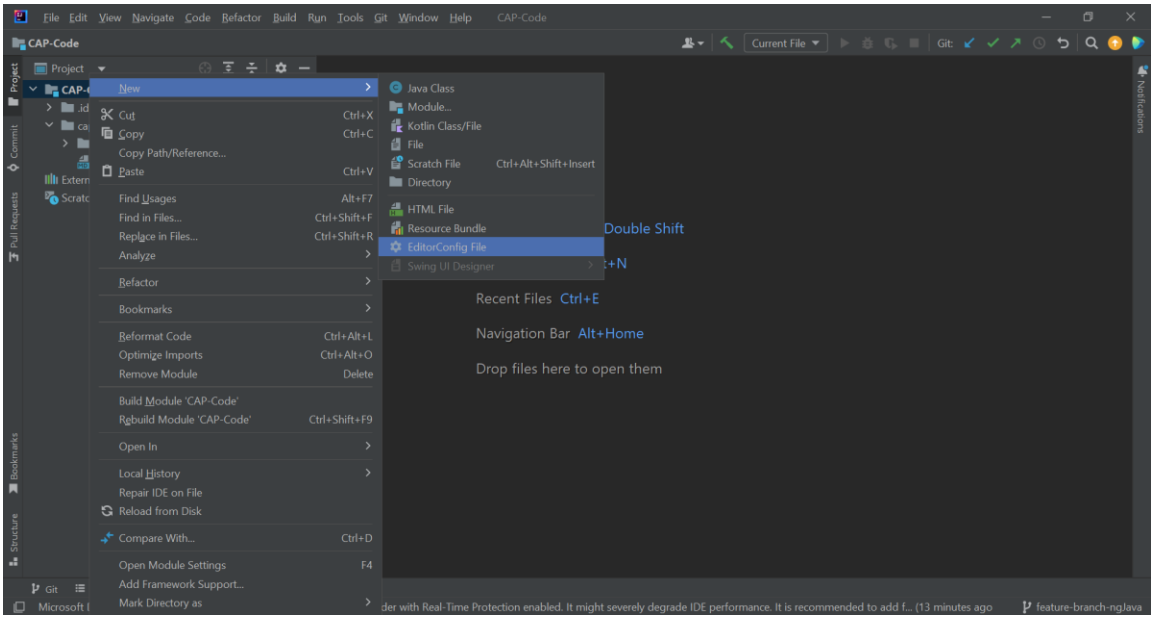
EditorConfig helps preserve consistent coding styles for multiple developers working on the same project across various IDEs.
It consists of a file format for defining coding styles and a collection of text editor plugins that can enable editors to read the file format and adhere to identified styles. EditorConfig files are easily readable, and they work nicely with version control systems.

In IntelliJ IDEA, you can configure indents in code style schemes or in .editorconfig files.

To be able to use EditorConfig in the project, the EditorConfig plugin should be enabled.

2.3.1 Steps to configures editorconfig in IntelliJ IDEA:

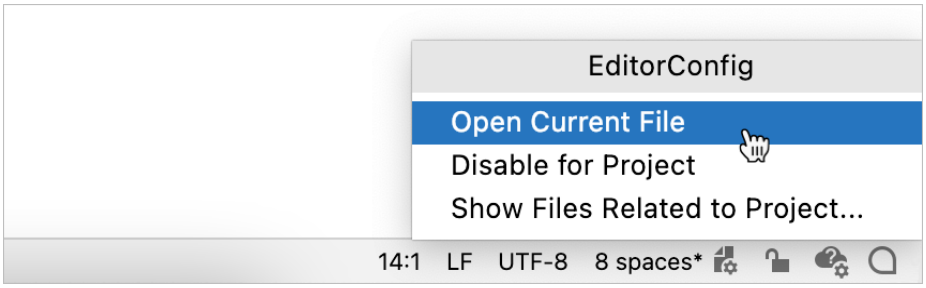
- In the Project tool window (Alt+1), right-click a source directory containing the files whose code style you want to define and select New | EditorConfig from the context menu.



- Select the properties that you want to define so that IntelliJ IDEA creates stubs for them, or leave all checkboxes empty to add the required properties manually
- Use code completion while filling .editorconfig files
- To preview how changes to your code style settings will impact the actual source files, click “eye icon” in the gutter of the .editorconfig file and select a source file on which you want to preview changes. The preview will open on the right.

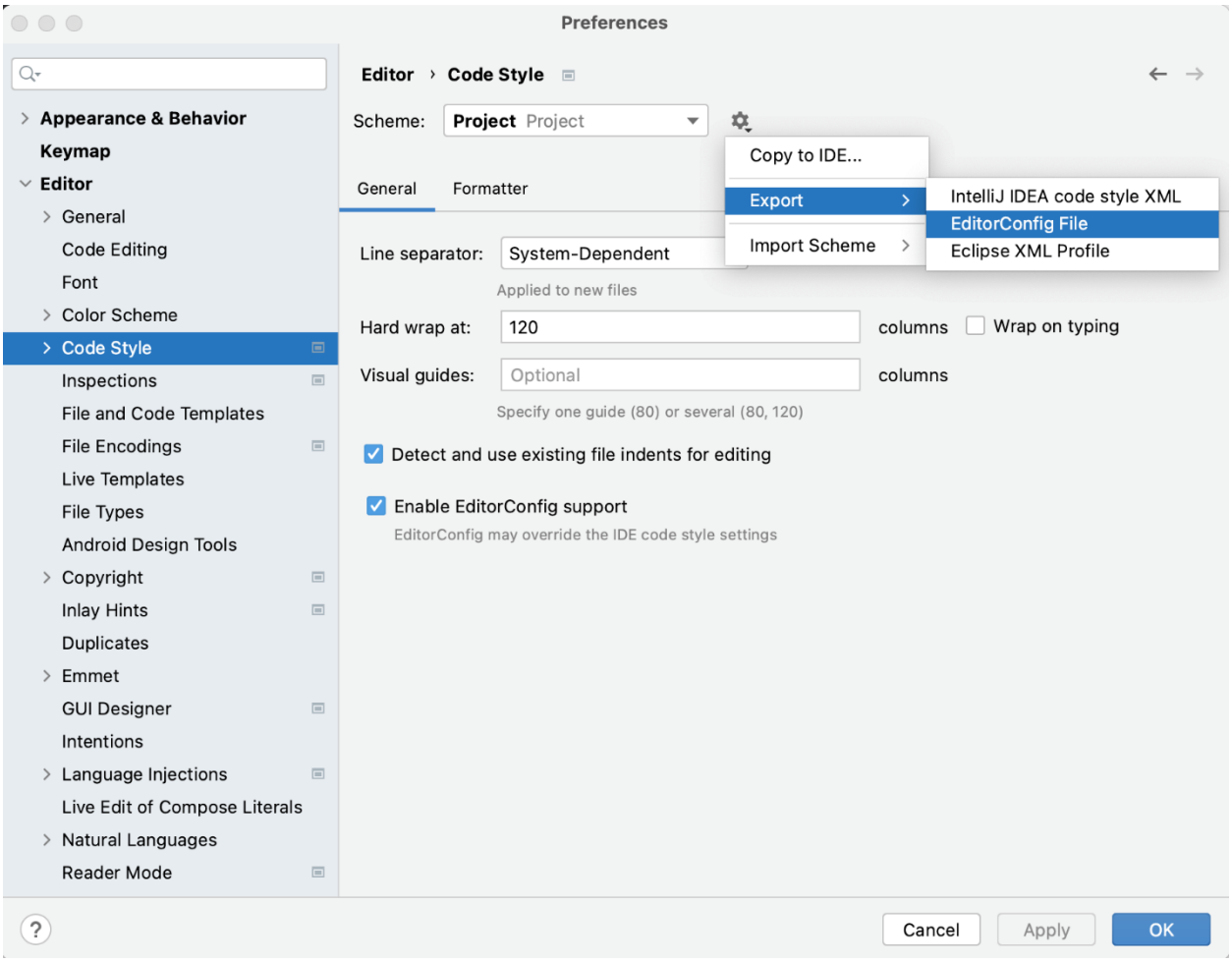
Click the widget:

- Click Open .editorconfig to open the nearest .editorconfig file which affects the file that you're currently working with
- Select Show Files Related to Project to open a list of all **.editorconfig** files in the project.
- Click Disable for Project to disable EditorConfig support in your project and use settings from your current code style scheme. You can also disable EditorConfig support in settings.



Export code style to an .editorconfig file

- Press Ctrl+Alt+S to open the IDE settings and select Editor | Code Style.
- Select the code style Scheme that you want to export: the Project scheme or one of the IDE-level schemes.
- Click Settings, select Export and then EditorConfig File.



- Specify the folder to which you want to save the file.
- To import code style settings in an .editorconfig file, drag the file to the necessary folder in the Project tool window (Alt+1 or View | Tool Windows | Project).

2.3.2 Indentations using EditorConfig

- To use tab indentation with a tab size of 4, the UTF-8 character set, and trim any trailing whitespaces in the code, we will define the following properties in the .editorconfig file

```
# Topmost editor config file
root = true

# Custom Coding Styles for Java files
[*.java]

# The other allowed value you can use is space
indent_style = tab

# You can change this value and set it to
# many characters you want your indentation to be
indent_size = 4

# Character set to be used in java files.
charset = utf-8
trim_trailing_whitespace = true
```

- EditorConfig applies styles in a top down fashion, so if we have several .editorconfig files in our project with some duplicated properties, the closest .editorconfig file takes precedence.

To apply the EditorConfig styles to all Java classes in our IntelliJ project, we click on the *Code* tab and select *Reformat Code* from the list of options. A dialog box should appear, and we can click on the *Run* button to apply our style changes.

2.3.3 Whitespaces

- Specify Whitespace Settings: In the .editorconfig file, define the whitespace settings you want to enforce.
- For Java projects, you can specify options like indent_style, indent_size, end_of_line, etc.

```
root = true

# Set default indentation style and size (use spaces, 4 spaces for indentation)

[*]

indent_style = space

indent_size = 4

# Set different settings for specific file types

[*.java]

# Trim trailing whitespace

trim_trailing_whitespace = true

# Ensure a new line at the end of the file

insert_final_newline = true

# Set the end of line (EOL) character to LF (Unix-style)

end_of_line = lf
```

3 SpringBoot

3.1 Spring Starter Packages

- **spring-boot-starter-data-rest -** The spring-boot-starter-data-rest is a Spring Boot starter that provides a quick and easy way to build RESTful APIs for your data entities. It allows you to expose your JPA entities (and other data sources) as RESTful endpoints with minimal configuration.
- **spring-boot-starter-actuator** – The spring-boot-starter-actuator is a Spring Boot starter that provides production-ready features to monitor and manage your Spring Boot application. It offers various endpoints and utilities to help you understand how your application is performing and to gather information about the application's health, metrics, configuration, environment, etc.
- **spring-boot-starter-data-jpa –** Provides dependencies for working with Java Persistence API (JPA) and enables database access and ORM.
- **spring-boot-starter-web -** Includes dependencies to build web applications with Spring MVC, Tomcat as the embedded web server, and other web-related components.
- **spring-boot-starter-security -** Offers dependencies to set up security features, including authentication and authorization.

3.2 POM

The Project Object Model (POM) file is an essential configuration file used in Apache Maven-based projects. It defines the project's metadata, dependencies, build settings, and other configurations.

3.2.1 POM best practices

- Keep the POM file well-formatted and easy to read. Practice proper indentation and line breaks to make it more readable, especially for larger projects.

- Consistent Dependency Versions: Use dependency management to ensure that all dependencies in the project have consistent versions. This helps avoid version conflicts and makes it easier to manage dependencies across the project.

- Avoid Using Wildcard Versions: Avoid using wildcard versions (e.g., 1.0.* or 1.*) for dependencies. Instead, specify the exact version or use the latest stable version available.

- Avoid Using System Scope: Avoid using the system scope for dependencies. The system scope is not recommended because it bypasses the normal dependency resolution process and can lead to portability issues.

- Use Dependency Management: Utilize the <dependencyManagement> section to define shared dependencies and their versions. This allows you to easily manage and update dependencies for the entire project.

- Override Property Defaults: If you use properties to define version numbers or other configuration values, ensure that the properties are defined in the correct order. Overriding property defaults should be done in the parent POM or early in the POM hierarchy.

- Avoid Duplicate Dependencies: Check for duplicate dependencies in the POM file. Having multiple dependencies for the same library and version can lead to unnecessary conflicts and increase the project's build time.

- Use Plugins Sparingly: Be cautious when adding plugins to the POM file. Limit the number of plugins to only those that are essential for the project. Too many plugins can lead to slower build times and unnecessary complexity.
- Separate Profiles for Different Environments: Use Maven profiles to define configurations for different environments (e.g., development, testing, production). This allows you to activate the appropriate profile based on the build environment.
- Avoid Hardcoding Paths: Avoid hardcoding absolute paths in the POM file. Instead, use Maven properties or environment variables for path configurations that may vary between different development environments.
- Define a Parent POM: If you have multiple projects in a multi-module Maven project, consider creating a common parent POM to centralize shared configurations and dependencies. This promotes consistency across modules.
- Use Plugin Management: For plugins that are used across different projects, define them in the <pluginManagement> section of the parent POM. This ensures that the same version and configuration are used consistently across all projects.

3.2.2 Cleanup POM File

```
mvn dependency:analyze
```

It shows you all dependencies that are in the POM, but that are not used in the source code. It also lists transitive dependencies which you accidentally used as direct dependencies.

3.3 Lombok

Lombok is a Java library that aims to reduce boilerplate code, particularly in POJO (Plain Old Java Object) classes. It provides annotations that generate common methods (e.g., getters, setters, toString, equals) during compile-time, thus reducing the need for developers to write these methods manually.

3.3.1 Annotations

- **@Data:** Generates getters, setters, equals, hashCode, and toString methods.
- **@Value:** It is used to create immutable classes with getter methods, equals, hashCode, and toString implementations, as well as private final fields.
- **@Getter / @Setter:** Generates getter or setter methods for individual fields.
- **@NoArgsConstructor / @AllArgsConstructor:** Generates constructors with no arguments or all arguments, respectively.
- **@RequiredArgsConstructor:** Generates a constructor with required fields as arguments.
- **@Builder:** Implements the builder pattern for the annotated class.
- **@ToString:** Generates a toString method.

By using Lombok annotations, we can keep the POJO classes clean and focused on the data represented, reducing the amount of boilerplate code.

Example of a simple Lombok-annotated class:

```
import lombok.Data;

@Data
public class Person {
    private String firstName;
    private String lastName;
    private int age;
}
```

With the @Data annotation, Lombok will automatically generate getters, setters, equals, hashCode, and toString methods for the Person class.

4 Logging

- **Use Logging Libraries:** Instead of using System.out.println() or System.err.println() statements, use a logging framework like Log4j, SLF4J, or java.util.logging (JUL). These libraries provide more sophisticated logging features and are easily configurable.

- **Choose Appropriate Log Levels:** Logging frameworks support different log levels (e.g., DEBUG, INFO, WARN, ERROR). Choose the appropriate level for each log statement. For example:

```
DEBUG: Detailed information, useful for debugging.

INFO: Important application events that can be helpful during troubleshooting.

WARN: Potential issues that might cause problems but can be handled gracefully.

ERROR: Critical errors that require immediate attention.
```

- **Avoid Logging Sensitive Information:** Do not log sensitive information like passwords, credit card numbers, or personal data. Use placeholders or redaction techniques if you need to log certain values.

- **Parameterized Logging:** Use parameterized logging to improve performance and avoid unnecessary string concatenation when constructing log messages. This can be done using placeholders or the "{}" syntax available in most logging frameworks.

```
Example of parameterized logging :

String username = "CBO";

int userId = 12345;

logger.info("User '{}' with ID '{}' logged in successfully.", username, userId);
```

- **Enable Logging Configurations:** Use external configuration files (e.g., properties, YAML) to configure logging behavior. This allows you to adjust log levels, log file locations, and other settings without modifying the code.

- **Contextual Logging:** Include contextual information in log messages, such as thread IDs, timestamps, and session IDs. This can be useful for tracing the flow of execution.

5 Exception Handling

Properly managing exceptions can improve the overall quality of your code and help identify and resolve issues effectively. Below are few best practices for exception handling in Java:

- **Specific Exceptions:** Use specific exception types rather than general ones like Exception or Throwable. This practice helps to clearly communicate the cause of the problem and makes it easier to handle different exceptional situations distinctly.

- **Keep Catch Blocks Short:** Put only the necessary code inside catch blocks. Long catch blocks may make the code harder to read and maintain.

- **Logging:** Always log exceptions and the relevant information such as stack traces, error messages, and input parameters. Logging helps in debugging issues and monitoring the application's health.

- **Avoid Catching Throwable:** Avoid catching Throwable directly unless you have a very compelling reason. Catching Throwable can also catch Error instances, which generally should not be caught and might lead to unexpected behavior.

- **Use Finally Block:** The finally block is used to ensure that code is executed whether an exception is thrown or not. Use it to release resources or perform cleanup operations.

- **Custom Exceptions:** Consider creating custom exception classes for specific error scenarios. This can enhance code readability and maintainability.

  Note : Need to create Custom Exception for the package structure

- **Don't Swallow Exceptions:** Avoid swallowing exceptions, which means catching an exception and not taking any action or not logging it. At least log the exception so that it can be investigated and fixed.
- **Follow a structure:** Log exceptions at their origin. If it’s a fail-safe scenario, do not throw the exception. If it’s a fail-fast scenario, throw a wrapped exception with the stack trace as cause and custom message for readability and accountability.

- **Handle at Service and Respond at Controller:** Handle all business scenarios in the Service class and send the response/failure to Controller class for it to redirect based on the response.

6 JUnits best practices

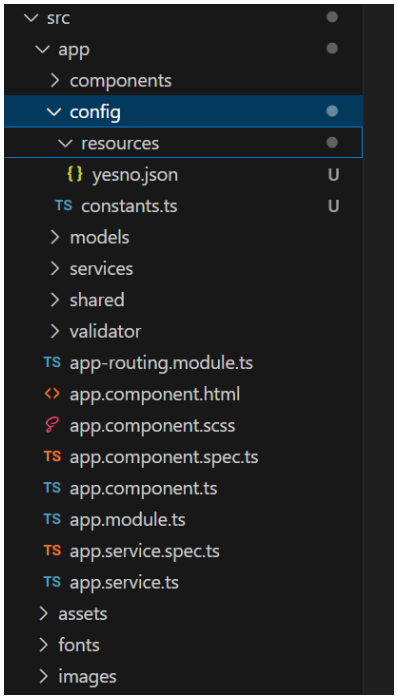
- **Isolate Tests:** Ensure that each test is independent and doesn't depend on the state of other tests. This prevents test failures due to interactions between tests and makes debugging easier.
- **Use Descriptive Test Names:** Give meaningful and descriptive names to your test methods so that it's clear what each test is verifying. This makes it easier for everyone to understand the purpose of each test.
- **Avoid Global State:** Avoid using global variables or shared state between test methods. Each test should have its own setup and teardown if required.
- **Use Test Fixtures:** Use @Before, @After, @BeforeClass, and @AfterClass annotations to set up and clean up test fixtures. These methods are run before and after each test or the entire test class.
- **Use Assertions Correctly:** Use appropriate assertion methods provided by JUnit (assertEquals, assertTrue, assertFalse, etc.) based on what you are trying to verify.
- **Keep Tests Fast:** Make sure your tests run quickly to encourage running them frequently. Slow tests can discourage developers from running them regularly, reducing their effectiveness.
- **Organize Tests:** Organize your test methods logically within the test class. Group related tests together and use nested classes if needed.
- **Use Parameterized Tests:** Use @ParameterizedTest to run the same test logic with different input parameters, which can help reduce code duplication.
- **Use Test Suites:** For larger projects, create test suites to organize and run related test classes together.
- **Mock External Dependencies:** Use mocking libraries like Mockito to isolate your code from external dependencies during testing.
- **Test Exception Scenarios:** Verify that methods throw the expected exceptions when provided with invalid inputs or when certain conditions are not met.
- **Maintain Code Coverage:** Aim to achieve good code coverage with your unit tests, but focus on testing critical and complex parts of your codebase.
- **Run Tests Frequently:** Run your tests frequently during development to catch issues early and ensure that your codebase remains stable.
- **Continuous Integration:** Integrate your tests into your Continuous Integration (CI) pipeline to ensure that tests are automatically executed whenever changes are pushed.
- **Update Tests with Code Changes:** Keep your tests up-to-date with code changes to ensure they remain valid and useful as your code evolves.

7 Angular Best Practices

- **Folder Structure:** Organize your project files in a logical folder structure, separating components, services, modules, and assets. This makes it easier to navigate and maintain the codebase.

  Following are some examples Angular Folder Structure:

  - app/components: Contains all the reusable components of the application.
  - app/services: Houses all the services that provide business logic and data access.
  - app/models: Handles all the models that maintains the request/response structure.
  - app/validator: Handles all kinds of validations performed on the forms.
  - app/modules: Organizes feature modules, each containing related components, services, and routing.
  - app/shared: Holds shared components, directives, and pipes used across the application
  - app/config: Contains reusable and immutable resources used across forms, constant files etc.,
  - app/core: Contains core functionality like guards, interceptors, and common services.
  - app/app.component.*: The root component files.
  - app/app.module.ts: The root module of the application.
  - assets/: Contains static assets like images, fonts, etc
  - environments/: Contains environment-specific configuration files.



This structure promotes a clear separation of concerns and allows easy navigation and maintenance. However, feel free to adjust it based on your project's specific needs and requirements. Also, adhere to the "Single Responsibility Principle," meaning each folder should have a clear purpose, and components, services, etc., should be placed in their respective folders to ensure maintainability and readability.

- **Use Angular CLI:** Utilize Angular CLI to create new projects, generate components, services, modules, etc. It automates many tasks and ensures a consistent project structure.
- **Component-Based Architecture:** Follow the component-based architecture for building Angular applications. Each component should have a single responsibility and be reusable.
- **Lazy Loading**: Employ lazy loading to load modules only when needed, reducing initial load times and improving performance.
- **Use Angular Modules:** Split your application into feature modules and shared modules. Feature modules group components, services, and other related entities, while shared modules contain reusable components and services used across the application.
- **Services for Business Logic**: Use services to encapsulate business logic and share data between components. Services promote reusability and maintain a clean separation of concerns.
- **Data Binding**: Prefer one-way data binding ([property] syntax) where possible to avoid unnecessary change detection cycles.
- **RxJS for Data Handling:** Utilize RxJS for handling asynchronous data and events. It provides powerful tools like Observables and operators for managing data streams.
- **Avoid Direct DOM Manipulation**: Leverage Angular's template-driven approach rather than direct DOM manipulation. Use built-in directives like *ngIf, *ngFor, etc.
- **Strict Mode**: Enable Angular's strict mode (strict: true in tsconfig.json) to enforce strict type-checking and catch potential errors early.
- **Use TypeScript:** TypeScript is the recommended language for Angular development. It provides static typing and helps catch errors during development.
- **AOT Compilation:** Use Ahead-of-Time (AOT) compilation to generate smaller, faster-loading bundles and detect errors before deployment.
- **Error Handling:** Implement a global error handling mechanism to gracefully handle errors that occur during runtime.
- **Testing:** Write comprehensive unit tests and end-to-end tests using tools like Jasmine and Karma to ensure code quality and avoid regressions.
- **Performance Optimization:** Pay attention to performance aspects like lazy loading, optimizing change detection, and using trackBy with *ngFor to minimize rendering overhead.
- **Code Linting and Formatting:** Use tools like SonarLint to enforce consistent coding styles and formatting across your project.

8 React Best Practices

- **Use Functional Components:** Prefer using functional components with hooks over class components. Functional components are easier to read, test, and maintain.
- **State Management:** For complex state management needs, consider using state management libraries like Redux or React's built-in Context API. For simpler state requirements, local state and hooks should suffice.
- **JSX and Components:** Keep your components small, focused, and reusable. Use JSX to write expressive and readable code.
- **Props and PropTypes:** Use props to pass data from parent to child components. When necessary, define PropTypes to validate the type and presence of props during development.
- **Controlled Components:** Use controlled components for form inputs, where the component state drives the input values.
- **Fragments:** Use React Fragments (<>...</>) or the shorthand syntax (<React.Fragment>...</React.Fragment>) to group multiple components without adding extra elements to the DOM.
- **Avoid Direct DOM Manipulation:** Use React's virtual DOM and avoid direct DOM manipulation whenever possible. Manipulating the DOM directly can cause side effects and make your code harder to reason about.
- **Component Lifecycle:** Understand the component lifecycle, but prefer using hooks like useEffect instead of class-based lifecycle methods for managing side effects.
- **Performance Optimization:** Optimize rendering performance by using React.memo, useMemo, useCallback, and avoiding unnecessary re-renders.
- **Code Splitting:** Consider code splitting to load only the required components for a particular route or feature, reducing the initial load time.
- **Error Boundaries**: Use error boundaries to catch and handle errors that occur within your components, preventing crashes in your application.
- **File Organization:** Organize your project files logically based on components, containers, services, etc., to enhance code maintainability.
- **Testing:** Write comprehensive unit tests using tools like Jest. Test your components' functionality to ensure reliability.
- **SonarLint**: Configure SonarLint to enforce consistent coding styles and formatting across your project.
- **Version Management:** Keep React and its dependencies up-to-date. Regularly update to the latest stable version to access new features and bug fixes.
- **Documentation:** Document your code, especially custom components and utility functions. Clear documentation makes it easier for other developers to understand and use your code.
- **Avoid Unnecessary Abstractions:** Don't overcomplicate your code with unnecessary abstractions or patterns. Keep it simple and maintainable.